

TP IA-GL

Systèmes de recommandation : de la popularité au filtrage collaboratif/basé sur le contenu

Imen Ben Sassi

imen.ben-sassi@umontpellier.fr

Objectif du TP

L'objectif de ce TP est de comprendre et de mettre en œuvre différentes approches de recommandation de dépôts GitHub à partir des interactions entre utilisateurs et dépôts.

À l'issue de ce TP, vous serez capables de :

1. construire une baseline de recommandation par popularité
2. construire une matrice utilisateur-dépôt à partir d'interactions implicites
3. implémenter un système de recommandation par
 - filtrage collaboratif basé sur les utilisateurs (user-based collaborative filtering)
 - filtrage collaboratif basé sur les objets (item-based collaborative filtering)
 - ou filtrage basé sur le contenu (content-based filtering)
4. comprendre la différence entre filtrage collaboratif et filtrage basé sur le contenu
5. évaluer un système de recommandation à l'aide de métriques Top-K
6. comparer les différentes approches et identifier leurs avantages et leurs limites.

Jeux de données

Les données fournies correspondent à une extraction de données GitHub. Trois fichiers sont disponibles :

- `github-dataset.csv` : métadonnées des dépôts (*stars_count*, *forks_count*, *language*, etc.)
- `users_stars.csv` : informations sur les utilisateurs (*location*, *company*, *used_languages*, etc.)
- `user_repo_stars_filtered_min_10.csv` : interactions entre utilisateurs et dépôts (avec au minimum 10 interactions par utilisateur). Une ligne indique qu'un utilisateur a donné une étoile à un dépôt.

Attention : l'absence d'une étoile ne signifie pas nécessairement que l'utilisateur n'aime pas le dépôt.

Étape 1 — Modèle basé sur la popularité

Un exemple de modèle simple est fourni en format Notebook *TP-IA-GL.ipynb* :

```
popular_repos = merged_repos.sort_values(by="stars_count", ascending=False)
```

Ce modèle recommande les dépôts les plus populaires à tous les utilisateurs, en excluant ceux qu'ils ont déjà étoilés.

À faire

- a) Exécuter et comprendre le code du modèle de popularité
- b) Identifier ses limites
- c) Évaluer le modèle à l'aide des métriques pour $K \in \{5, 10, 15, 20\}$:
 - Precision@K
 - Recall@K
 - F1@K
- d) Répondre aux questions :
 - Quels types de dépôts sont favorisés ?
 - Quels utilisateurs peuvent être pénalisés par cette approche ?
 - Comment ce modèle se comporte-t-il pour un utilisateur ayant des goûts très spécifiques ?

Cette méthode constitue votre **baseline**.

Étape 2 — Implémentation d'une autre approche

À partir de cette baseline, implémentez l'une des approches suivantes :

A. Filtrage collaboratif basé sur les utilisateurs

L'idée est la suivante : *Des utilisateurs ayant des historiques d'interactions similaires sont susceptibles d'avoir des préférences similaires.*

- Construisez une matrice utilisateur–objet (binaire : 1 si l'utilisateur a étoilé le dépôt et 0 sinon)
- Calculez la **similarité entre utilisateurs** (avec la similarité cosinus)
- Recommandez à un utilisateur les dépôts les plus populaires parmi ses **voisins les plus similaires**.
- Répondez aux questions suivantes :
 - Quel est l'effet du nombre de voisins ?
 - Que se passe-t-il lorsqu'un utilisateur possède très peu d'interactions ?
 - Les utilisateurs les plus actifs obtiennent-ils de meilleures recommandations ?

B. Filtrage collaboratif basé sur les objets

L'idée est la suivante : *Cette fois, on ne recherche plus des utilisateurs similaires mais des dépôts similaires. Deux dépôts sont considérés comme similaires lorsqu'ils sont souvent étoilés par les mêmes utilisateurs.*

- Construisez une matrice utilisateur–objet (binaire : 1 si l'utilisateur a étoilé le dépôt et 0 sinon)
- Calculez la **similarité entre dépôts** selon les utilisateurs qui les ont étoilés
- Recommandez à un utilisateur les dépôts **les plus similaires à ceux qu'il a déjà étoilés**.
- Testez différentes valeurs du nombre de voisins utilisés pour chaque dépôt : $N \in \{3, 5, 10, 20\}$

C. Filtrage basé sur le contenu

L'idée est la suivante : *Contrairement au filtrage collaboratif, le content-based filtering utilise les caractéristiques des dépôts plutôt que les interactions des utilisateurs.*

- Représentez chaque dépôt sous forme de vecteur de caractéristiques (*language, stars_count, forks_count*, etc.).
Vous pourrez par exemple utiliser :
 - One-Hot Encoding pour les variables catégorielles
 - une normalisation pour les variables numériques
- Calculez la **similarité cosinus** entre les dépôts
- Recommandez à un utilisateur les dépôts les plus **similaires à ceux qu'il a déjà étoilés**.

Étape 3 — Évaluation et comparaison

Pour chaque utilisateur, le système produit une liste de K recommandations. On considère comme pertinentes les recommandations correspondant aux dépôts présents dans le jeu de test de cet utilisateur.

- Réutilisez les mêmes métriques (Precision@K, Recall@K, F1@K)
- Comparez vos résultats à ceux du modèle de popularité en variant $K \in \{5, 10, 15, 20\}$
- Discutez :
 - Quels utilisateurs bénéficient le plus de chaque approche (par rapport à son profil : nombre d'interactions, nombre de langages utilisés, etc.) ?
 - Les systèmes ont-ils tendance à recommander toujours les mêmes dépôts ? Que se passe-t-il lorsqu'un nouvel utilisateur n'a encore aucune interaction ?
 - Que se passe-t-il lorsqu'un nouveau dépôt vient d'être créé et n'a encore reçu aucune étoile ?
 - Quels avantages et limites observez-vous ?

Livrable attendu

Le livrable est constitué de deux éléments :

- Le code Python commenté de votre approche, présenté sous forme de notebook, incluant l'exécution et l'affichage des résultats pour chaque cellule
- Des slides présentant votre travail :
 - la méthode implémentée
 - les résultats obtenus
 - la comparaison avec le modèle de popularité
 - une analyse critique des résultats.